

Breaking the Scalability Barrier in Constrained Graph-Based Networked Control via Decision-Focused Learning

Zhaoxing Yang
Shanghai Jiao Tong University
Shanghai, China
yiannis@sjtu.edu.cn

Yuchen Guo
Shanghai Institute of Satellite
Engineering
Shanghai, China
gycsatellite@126.com

Wenlong Li
Shanghai Institute of Satellite
Engineering
Shanghai, China
liwenlongzacao@126.com

Guiyun Fan*
Tongji University
Shanghai, China
fanguiyun@tongji.edu.cn

Haiming Jin
Shanghai Jiao Tong University
Shanghai, China
jinhaiming@sjtu.edu.cn

Linghe Kong
Shanghai Jiao Tong University
Shanghai, China
linghe.kong@sjtu.edu.cn

Abstract

Many real-world systems can be modeled as graphs, where nodes store and consume entities, actively produce them, or have them emerge naturally, and edges transport them between nodes. This paper studies the networked control problem on such large-scale systems, aiming to decide production and transportation over time to maximize long-term profits, subject to node or edge capacity constraints. Existing SOTAs either fail to guarantee feasibility or cannot scale to large-scale systems. We propose a two-stage policy that integrates a constrained optimization layer after a neural network to explicitly enforce constraints and ensure feasibility. By leveraging the problem structure to obtain expert actions and designing a decision-focused and differentiable loss to enable imitation learning, our method significantly improves efficiency and scalability. In small-scale systems with action dimensions in the order of 10, our method achieves 60x sample efficiency over SOTAs on average. In large-scale systems with action dimensions ranging from 100 to 100000, where SOTAs fail to train, our method converges quickly and outperforms non-learning-based baselines significantly.

CCS Concepts

• **Mathematics of computing** → **Graph algorithms**; • **Computing methodologies** → *Sequential decision making*.

Keywords

Networked Control, Graph Algorithm, Decision-Focused Learning

ACM Reference Format:

Zhaoxing Yang, Yuchen Guo, Wenlong Li, Guiyun Fan, Haiming Jin, and Linghe Kong. 2026. Breaking the Scalability Barrier in Constrained Graph-Based Networked Control via Decision-Focused Learning. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792331>

*Corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License. *WWW '26, Dubai, United Arab Emirates*
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792331>

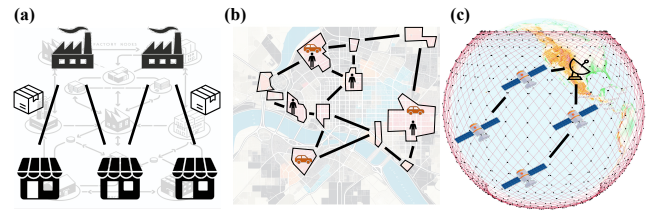


Figure 1: Illustration of real-world systems: (a) SCIM, (b) DVR, (c) SDT. (a-b) are partially generated by DALL·E 3.

1 Introduction

Many real-world systems consist of large-scale nodes connected by edges. At each node, entities can be produced, randomly emerge, be stored, or be consumed; additionally, entities can be transported between nodes. The consumed entities generate revenue, while production, storage, and transportation of entities incur expenses. Node storage or edge capacity is usually constrained. Examples of such systems include Supply Chain Inventory Management (SCIM), Dynamic Vehicle Routing (DVR) and Satellite Data Transmission (SDT), as depicted in Fig. 1. In SCIM, the entities are goods, produced by factory nodes and transported to hundreds of merchant nodes to meet customer demand. In DVR, the entities are idle vehicles, which appear randomly and are dispatched between hundreds of city stations to serve passenger orders. In SDT, the entities are observation data packets, which emerge at thousands of satellite nodes and are transmitted to ground stations for monitoring tasks.

In these systems, a fundamental problem is *how to decide the production and transportation of entities under system constraints to maximize long-term profits*. We formulate the problem as a general Constrained Networked Control Problem (CNCP), which naturally corresponds to a capacity-constrained dynamic optimization problem on graphs. Through a survey of existing works, we found that state-of-the-art (SOTA) methods for solving CNCP can be categorized into the following two types, each with critical limitations in feasibility, scalability, or optimality.

The first type of method relies solely on neural network (NN) modules [6, 23]. Their core idea is to use powerful diffusion models to fit the distribution of feasible actions based on offline datasets containing such feasible actions. In testing, rewards are used to guide the diffusion models to generate actions to optimize rewards. However, in our pilot experiments with more than 3000 hyper-parameter combinations, the diffusion models struggle to satisfy

constraints. We believe this is because the constraints in CNCP are coupled and involve decisions across multiple nodes. Each node's decision must account for its impact on other nodes and the potential for constraint violations. As a result, NN modules alone are not well-suited for this task, leading to frequent constraint violations.

The second type of method uses NN module and constrained optimization solving module as the policy [10, 18]. Firstly, NN module processes the input state and generates intermediate outputs. Then, these intermediate outputs are input to a constrained optimization problem to produce feasible solution as the action. Reinforcement Learning (RL) algorithm is used to optimize the NN module, and it has shown good performance in small-scale systems (e.g., action dimensions in the order of 10). However, it struggles to scale and performs poorly in higher-dimensional action spaces (e.g., action dimensions over the order of 100), even falling short of simple heuristics. We believe the reason is that the exploration efficiency of RL gets significantly deteriorated facing high-dimensional continuous action spaces, making it difficult to explore and converge.

Hence, we aim to propose a scalable method for CNCP while satisfying constraints. Specifically, our method also uses a two-stage policy, allowing constraints imposed explicitly in the second-stage optimization to ensure feasibility. To optimize the first-stage NN module, we propose a decision-focused imitation learning algorithm (DFIL). DFIL leverages expert decisions, which are overlooked by SOTAs, to provide optimal and unbiased update signals, enabling efficient and scalable training. The core of DFIL is a decision-focused, consistent, and differentiable loss that explicitly considers the second-stage optimization problem. This ensures the loss focuses on the optimality of the second-stage output actions rather than the intermediate results output by the first-stage NN module.

In summary, our paper has four-fold main contributions.

- To the best of our knowledge, we are the first to motivate and formulate the general Constrained Networked Control Problem (CNCP). Thus, our DFIL algorithm provides an efficient solution for a broad class of problems that are reducible to CNCP.
- We identify and validate the limitations of SOTA methods, including their inability to satisfy constraints and scale effectively. These limitations motivate our policy and algorithm design.
- We propose a two-stage policy that ensures action feasibility and introduce a key decision-focused, consistent, and differentiable loss to enable IL for scalable optimization.
- DFIL achieves an average 60x improvement in sample efficiency over SOTAs in small-scale systems with action dimensions in the order of 10. In large-scale systems with action dimensions over 100 and even 100000, it quickly converges to strong performance, whereas SOTAs fail to train at all.

2 Problem Description

We begin by introducing the CNCP we aim to solve. Then, we show instantiations of CNCP on three real-world applications, and discuss limitations of SOTA methods.

2.1 Problem Formulation

Basics. The time horizon of CNCP is divided into T equally-sized time steps, denoted by set $\mathcal{T} = \{1, \dots, T\}$. Let $\mathcal{G}_t = (\mathcal{V}, \mathcal{E}_t)$ be

the graph of each t that contains the node set $\mathcal{V}$ and the edge set $\mathcal{E}_t \subseteq \{(i, j) | i, j \in \mathcal{V}\}$.

Events. At the beginning of each time step $t \in \mathcal{T}$, each node $i \in \mathcal{V}$ possesses a quantity $x_t^i \in \mathbb{R}_{\geq 0}$ of entities. x_t^i would evolve due to the following events.

- *Emergence.* New entities could be produced by nodes in some applications. If so, the controller has to make decisions on how much quantity $p_t^i \in \mathbb{R}_{\geq 0}$ to produce for i . Let $\tau \in \mathbb{N}_{\geq 0}$ be the known time steps to finish the production, i.e., such p_t^i quantity of entities will become available for i at step $t + \tau$. The expense required for unit production is q^{Pr} . Otherwise, new entities may randomly emerge at each node without the need for decision-making on production in other applications¹.
- *Transportation.* For any node pair (i, j) , entities could be transported from i to j along paths in the application-specific candidate path set $\mathcal{M}_{i,j,t}$. The controller has to make decisions on the portion of entities of i to be transported along each path $m \in \mathcal{M}_{i,j,t}, \forall i, j \in \mathcal{V}$ at the beginning of t , i.e., $f_t^{i,j}(m) \in [0, 1]$. We assume that it takes less than one time step to finish each transportation. The expense required for unit transportation is q^{Tr} .
- *Consumption.* Entities could be consumed due to randomly emerging demands d_t^i during step t . The revenue generated for unit consumption is q^{Cs} .
- *Storage.* The un-transported and un-consumed entities will stay in the node till the next step, which requires expense q^{St} for unit entity.

Profit. The profit of each step t is the revenue minus the expense of all nodes. Let $\bar{d}_t^i = \min\{d_t^i, x_t^i - x_t^i \cdot \sum_{j \in \mathcal{V}} \sum_{m \in \mathcal{M}_{i,j,t}} f_t^{i,j}(m)\}$ be the satisfied demands at i , then the profit R_t is defined as,

$$\sum_{i \in \mathcal{V}} \left(q^{\text{Cs}} \cdot \bar{d}_t^i - \sum_{j \in \mathcal{V}} \sum_{m \in \mathcal{M}_{i,j,t}} q^{\text{Tr}} \cdot x_t^i \cdot f_t^{i,j}(m) - q^{\text{Pr}} \cdot p_t^i - q^{\text{St}} \cdot (x_t^i - \bar{d}_t^i - x_t^i \cdot \sum_{j \in \mathcal{V}} \sum_{m \in \mathcal{M}_{i,j,t}} f_t^{i,j}(m)) \right).$$

Constraints. The following constraints are imposed for decisions. Firstly, all the decisions have to satisfy the following validness constraints for each step t ,

$$\sum_{j' \in \mathcal{V}} \sum_{m' \in \mathcal{M}_{i,j',t}} f_t^{i,j'}(m') \leq 1, \quad (\text{Validness})$$

$$p_t^i \geq 0, f_t^{i,j}(m) \geq 0, \forall i, j, m \in \mathcal{M}_{i,j,t}.$$

Secondly, the total quantity of entities passing an edge e is limited by its capacity c^e for each step t ,

$$\sum_{i,j \in \mathcal{V}} \sum_{\substack{e \in \mathcal{M} \\ m \in \mathcal{M}_{i,j,t}}} f_t^{i,j}(m) \cdot x_t^i \leq c^e, \forall e \in \mathcal{E}_t. \quad (\text{Edge Capacity})$$

The final constraint applies storage capacity c^i on the quantity of entities of node i for each step t ,

$$0 \leq x_t^i \leq c^i, \forall i. \quad (\text{Node Capacity})$$

¹For simplicity, we use a unified event to describe the appearance of new entities at a node, regardless of whether they are produced through controllable actions or arise randomly from external factors.

Problem to Solve. The controller aims to find the solution to the following problem to optimize the expected long-term profit under the diverse constraints introduced before:

$$\begin{aligned} & \text{maximize} \quad \mathbb{E} \left[\sum_{t=1}^T R_t \right] \\ & \text{subject to} \quad (\text{Validness}), (\text{Edge Capacity}), (\text{Node Capacity}), \forall t. \end{aligned} \quad (\text{CNCP})$$

In CNCP, the objective is complicated by its stochastic nature, as the emergence of entities (if production is unsupported) and demands are not only stochastic, but also unknown a priori, which makes directly solving CNCP infeasible.

Remark on the Scale. Although CNCP is theoretically applicable to graphs of arbitrary scale, in this paper we focus on graphs with up to several thousand nodes, as such sizes are already sufficient to capture large-scale real-world scenarios. In all three applications considered, our experiments involve graphs with thousands of nodes—systems that, to the best of our knowledge, are at or beyond the typical scales studied in the field, and already pose substantial challenges to SOTA methods, as demonstrated next in Sec. 2.3.

2.2 Real-world Applications

To make CNCP easier to understand, we instantiate CNCP on three real-world applications, which are introduced as follows and summarized in Table 3 in the appendix. Due to space limitations, the full specification of decision variables, constraints, and objectives for each application is provided in the Appendix B-D.

Supply Chain Inventory Management (SCIM). A SCIM system contains factory nodes and merchant nodes. Factory nodes produce goods and transport them to the merchant nodes to meet the customer demands there. Satisfied demands generate revenue, while the production, transportation and storage of goods generate expenses. The candidate path set contains the shortest path between nodes. The goal of SCIM is to decide the production and transportation quantity of goods to maximize long-term profits under the constraints of node storage capacity.

Dynamic Vehicle Routing (DVR). DVR divides the city into stations connected by roads. Passenger demands randomly appear at stations, and vehicles at the same station could pick up passengers and take them to their destinations. Serving passengers generates revenue, while dispatching idle vehicles between stations generates expenses. The candidate path set of DVR contains the shortest path between stations. The goal of DVR is to control the dispatching quantity of idle vehicles to maximize long-term profits.

Satellite Data Transmission (SDT). In SDT, earth observation satellites need to transmit observation data of the target region to ground stations via Inter-Satellite Links (ISLs) and Satellite-to-Ground Links (SGLs). These links are time-varying as they may get disrupted due to factors such as atmospheric orbit drags, multi-body orbit perturbations, solar/lunar activities, thermal fluctuations, and space radiations. The observation data is regularly generated at the satellites and consumed at the ground stations. The candidate path set of SDT consists of several shortest paths from satellite to the ground station. The goal of SDT is control the transmission

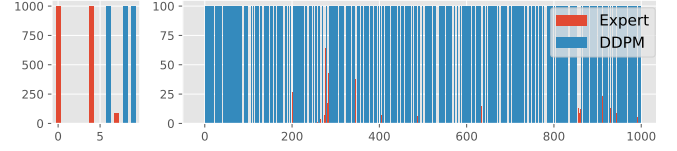


Figure 2: Expert and DDPM action. The action is 1010-d. Left: the first 10-d with clip range $[0, 1000]$. Right: the remaining 1000-d with clip range $[0, 100]$. Note that clipping operation is used only for the DDPM action to make it non-negative and within the same scale as the expert action.

quantity of data to maximize long-term throughput under the link bandwidth constraints.

2.3 SOTA Methods and Limitations

Based on the difference of the adopted policies, SOTA methods can be categorized into two types, each with its respective limitations: difficulty in satisfying constraints and difficulty in scalability. More comprehensive discussions on related works are shown in Sec. 8.

One-stage Policy. The policy of this type uses neural network models only. The SOTA methods, [6, 23], developed in physics and autonomous driving domains, utilize the powerful diffusion models to approximate the distribution of feasible actions from the offline dataset containing states and feasible actions. During inference, the Langevin sampling procedure, based on the gradient of the trained denoising network, is used to sample feasible actions.

However, in our pilot experiments, regardless of changes made to the number of steps for denoising, the dimension of hidden layers, batch size, learning rates, and so forth, the denoising network fails to meet the constraints. We have attempted over 3,000 combinations of hyperparameters, all of which ended in failure, as detailed in Appendix E. The primary reason for the failure is that the constraints in CNCP are coupled, with each constraint often involving decisions from multiple nodes. This coupling property makes the feasible actions highly discontinuous and non-smooth, which poses significant challenges for deep models to be effective. To make it clear, we generate dataset of expert actions from SCIM-10f100s system and train the denoising network using the DDPM algorithm [13]. Fig. 2 compares the action sampled by the denoising network and the expert action. It shows that expert action has sharp and steep changes in adjacent dimensions, and the denoising network completely fails to fit such action distribution.

Two-stage Policy. The policy of this type of methods, GRL [10, 18], adopts a two-stage decision-making pipeline. Firstly, a deep Neural Network (NN) policy is used, outputting intermediate decisions based on the current state. Then, such decisions are treated as parameters in a constrained optimization problem, whose solution is the action. GRL uses RL to optimize the first stage NN policy. It has achieved satisfactory results in small-scale systems, however, as observed in our experiments, the performance of GRL deteriorates in large-scale systems and GRL can even be challenging to train. This is due to the high-dimensional continuous action spaces in large-scale systems, making exploration difficult for RL algorithms.

3 Methodology

Given the failure of one-stage policy in satisfying constraints, this work adopts the two-stage policy, introduced in Sec. 3.2, as its

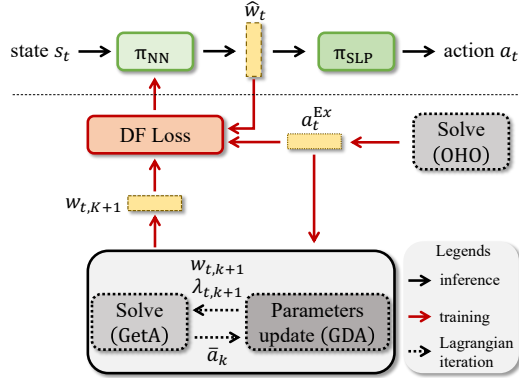


Figure 3: Our policy (top) and training method (bottom).

second-stage optimization problem could consider system constraints explicitly to ensure action feasibility. To efficiently and scalably optimize the first-stage NN policy module, we propose a decision-focused training method, detailed in Sec. 3.3. The whole policy framework and the training method is shown in Fig. 3. To ensure a rigorous presentation and precise use of notations, we firstly introduce the Instantaneous-Constrained Markov Decision Process (ICMDP) modeling of CNCP in Sec. 3.1.

3.1 ICMDP Formulation

We re-formulate the CNCP as a ICMDP $(\mathcal{S}, \mathcal{A}, C, R, P)$. Each component is defined as follows.

- $\mathcal{S}$ is the state space. For each time step $t \in \mathcal{T}$, $s_t = \{s_t^i\}_{i \in \mathcal{V}}$, and s_t^i is the local state of node i . In particular, $s_t^i = (x_t^i, \tilde{p}_t^i, \{d_t^{i,j}\}_{j=1}^{t-1})$, where $\tilde{p}_t^i = \{p_{t'}^i\}_{t'+t' \geq t}^{1 \leq t' < t}$ stands for the quantity of entities that are still in production. $\{d_t^{i,j}\}_{j=1}^{t-1}$ tracks historical demands, covering the entire time step from $t' = 1$ to $t - 1$. This is because demands may be non-Markovian, thus, we take into account the complete history of the realized demands.
- $\mathcal{A}$ denotes the action space. For each time step t , $a_t \in \mathcal{A}$ includes the production quantity of each node and the transportation portion of each path. We represent a_t in the form of vector for ease of subsequent use, $a_t = [\{p_t^i, f_t^{i,j}(m)\}_{i,j \in \mathcal{V}}^{m \in \mathcal{M}_{i,j,t}}]$. In systems where nodes do not support production, such as DVR and SDT, the action only includes the transportation portion.
- C represents the constraints each action a_t must satisfy, including the three constraints defined in CNCP: edge capacity, node capacity, and validness.
- The immediate reward R is the profit of the step.
- P represents the unknown transition function, as the underlying process that determines the emergence of demands is unknown.

Through the definition, we utilize ICMDP to describe the sequential decision-making process of CNCP. In this definition, we do not introduce additional assumptions, and the above ICMDP remains applicable even when the demands are non-Markovian. Therefore, this ICMDP is sufficiently universal. We also note that our ICMDP is orthogonal to the popular CMDP formulation where the long-term cost of full trajectory is constrained [2], while ICMDP imposes more strict constraints on the action of each step.

We denote the policy as π , which outputs action based on the state input, i.e., $a_t = \pi(s_t)$. Then, our goal is to find the optimal feasible policy π^* to maximize the long-term expected reward, which is equivalently to solve CNCP.

3.2 Two-stage Policy

Our policy π contains two stages. The first stage is an NN policy π_{NN} . It takes the state s_t as input, and outputs the weights $\hat{w}_t$ for the actions. The second stage is an optimization policy π_{SLP} , which solves a Linear Programming (LP) with surrogate objective to obtain the final action a_t .

Policy π_{NN} . In the first stage, π_{NN} takes the current state s_t at step t as input, and outputs weights $\hat{w}_t$ for prioritizing different action dimensions. When the demands are non-Markovian, sequence model like RNN is necessary for π_{NN} . Otherwise, the policy can be implemented with the widely used 3-layer ReLU network.

Policy π_{SLP} . In the second stage, the following Surrogate LP (SLP) is solved to obtain the action a_t .

$$\begin{aligned} & \text{minimize} \quad \hat{w}_t^\top a & (\text{SLP}) \\ & \text{subject to} \quad (\text{Validness}), (\text{Edge Capacity}), (\text{Node Capacity}) \text{ at } t. \end{aligned}$$

The reason for designing (SLP) in this form is threefold: firstly, it remains an LP problem due to linear objective and constraints, which allows for efficient solving; secondly, all the three constraints are free of $\hat{w}_t$ to ensure feasibility; thirdly, its concise objective is a classic form that has been extensively studied in the literature, facilitating the design of decision-focused learning methods.

Remark on Minimization. A potential source of confusion is that while our overall objective is to maximize long-term profit, the (SLP) involves minimizing a linear weighted value. This minimization is not intended for single-step profit maximization; rather, it serves as a tractable subproblem whose solutions are aligned with experts. Our decision-focused training explicitly optimizes π_{NN} so that (SLP) produces outputs matching experts, thereby indirectly driving long-term profit maximization. This approach avoids the pitfall of myopic single-step maximization and enables learning policies that perform well in long-term objectives.

3.3 Decision-Focused Imitation Learning

Key Idea. We achieve efficiency and scalability through an IL-based method, as it provides unbiased and optimal update signals for policy optimization. Compared to RL, which relies on self-exploration, this method enables more stable training and faster convergence. This advantage becomes particularly significant in large-scale systems, where the exploration efficiency of RL tends to deteriorate heavily.

Challenges and Organizations. To make IL viable, we need to address two challenges: obtaining expert actions and designing differentiable loss. Next, we show that by exploiting the structure of CNCP, we can obtain expert actions. We then introduce a differentiable loss under the decision-focused principle. Finally, we show how to obtain the inputs for the loss and summarize the algorithm.

3.3.1 Obtain Expert Actions. After interacting with the system for an episode, all the demands appearing in this episode become realized. Then, we can construct an offline constrained optimization problem which considers the long-term profit, with realized transitions, and solve it. The transportation portions and production quantities in the solutions are the expert actions.

Based on the realized demands, we construct the following transition equation for each step t .

$$x_{t+1}^i = x_t^i \cdot (1 - \sum_{j \in \mathcal{V}} f_t^{i,j}(m)) + p_{t+1-\tau}^i - \bar{a}_t^i + \sum_{j \in \mathcal{V}} f_t^{j,i}(m) \cdot x_t^j, \forall i. \quad (\text{Transition})$$

With the transitions, we construct the Offline H-Optimal (OHO) problem. For each t , it considers the transition of entity quantities, constraints, and total profit over the next H steps. By solving (OHO), we could obtain the solutions from t to $t+H-1$, and select the solution of t as the expert action a_t^{Ex} .

$$\begin{aligned} & \text{maximize} \quad \sum_{t'=t}^{t+H-1} R_{t'} \\ & \text{subject to} \quad (\text{Validness, Edge Capacity}), \forall t \leq t' < t+H, \\ & \quad (\text{Node Capacity, Transition}), \forall t \leq t' < t+H. \end{aligned} \quad (\text{OHO})$$

Remark on H. (OHO) considers the horizon $H < T$ instead of T . The reason for this is the potentially high computational complexity when considering the whole horizon T , as (OHO) involves nested transitions across time steps, which could potentially result in a nonlinear optimization problem, requiring more resources and time for solving. In fact, this involves a trade-off between offline efficiency and online efficiency. If the controller has sufficient offline resources, a larger H can be set, enabling the decision at each t to consider a longer-term future. Then, the expert actions obtained would be closer to the optimality, and the trained policy will also be closer to the optimality. Fig. 7 compares the performance of trained policies with different H . It can be observed that a larger H achieves higher performance, albeit at the cost of longer training time.

3.3.2 DF Loss Function. In this section, we introduce our DF loss function, assuming the ground truth weight vector w_t is known. We will show how to obtain it in the next section. Note that w_t is referred to as the ground truth because, when used as input to (SLP), it generates the expert action a_t^{Ex} . Furthermore, we avoid the more straightforward mean squared error (MSE) loss $\|w_t - \hat{w}_t\|_2^2$ because our goal is decision accuracy: $\hat{w}_t$ matters only through the (SLP)-induced action a_t , and reducing weight error does not necessarily improve a_t . Hence we use a decision-focused loss that differentiates through (SLP) to align gradients with action quality; the advantage over MSE is validated in Sec. 7. The loss under this goal is defined as follows.

Definition 3.1 (Direct Loss). Given a π_{NN} policy output $\hat{w}_t$ and a ground truth weight vector w_t , the direct loss is defined as $\ell_{\text{dir}}(\hat{w}_t, w_t) := w_t^\top \cdot \text{SLP}(\hat{w}_t) - w_t^\top \cdot \text{SLP}(w_t)$.

In the loss, $\text{SLP}(w_t)$ denotes the solution of (SLP) when w_t is the input, which is in fact a_t^{Ex} . $\text{SLP}(\hat{w}_t)$ is the solution when $\hat{w}_t$ is the input. Such loss exactly measures the error incurred when making a suboptimal action due to an imprecise weight outputted by π_{NN} .

Algorithm 1 Training Algorithm for π_{NN}

```

1: Initialize: buffer  $\mathcal{B}$  as empty  $\emptyset$ ; policy  $\pi_{\text{NN}}$ ; weight  $w_{t,0}$ , parameter  $\lambda_{t,0}$ ,  $\forall t$  as zero;
2: repeat
3:   for  $t = 1$  to  $T$  do
4:     Infer weight  $\hat{w}_t$  based on state  $s_t$  using  $\pi_{\text{NN}}$ ;
5:     Solve (SLP) to obtain action  $a_t$ ;
6:     Execute action  $a_t$  and obtain  $s_{t+1}, R_t$ ;
7:     Store  $(s_t, a_t, R_t, s_{t+1})$  into buffer  $\mathcal{B}$ ;
8:   end for
9:   With buffer  $\mathcal{B}$ , solving (OHO) for each  $t$  to get expert actions  $\{a_t^{\text{Ex}}\}_{t=1}^T$ ;
10:  For each  $t$ , update parameters with (GDA) and (GetA) for  $K$  iterations;
11:  Use loss  $\sum_t \ell_{\text{df}}(\hat{w}_t, w_{t,K+1})$  to update  $\pi_{\text{NN}}$ ;
12:  Set buffer  $\mathcal{B}$  as empty  $\emptyset$ ;
13: until  $\pi_{\text{NN}}$  converges

```

However, we cannot use this loss to optimize π_{NN} as the involved (SLP) is non-differentiable and may not be continuous in $\hat{w}_t$. To make training feasible, we turn to a surrogate loss, inspired by the classical SPO+ loss [7].

Definition 3.2 (DF Loss). Given a π_{NN} policy output $\hat{w}_t$ and a ground truth weight vector w_t , the loss is defined as $\ell_{\text{df}}(\hat{w}_t, w_t) := \max_{y \in S_t} \{w_t^\top y - 2\hat{w}_t^\top y\} + 2\hat{w}_t^\top a_t^{\text{Ex}} - z_t^*$.

In the definition, S_t is the feasible region that the three constraints, (Validness), (Edge Capacity) and (Node Capacity) co-generates at step t . z_t^* is the profit that the expert action a_t^{Ex} generates. As ℓ_{df} is linear to $\hat{w}_t$, such loss is differentiable to $\hat{w}_t$. Thus, it can be used to optimize π_{NN} . Another important property is shown below.

THEOREM 3.3 (ADAPT THEOREM 1 IN [7]). *Under mild assumptions, the DF Loss is Fisher consistent with respect to the Direct Loss.*

The Fisher consistency essentially means that the optimal π_{NN} also minimizes the DF loss, ensuring that optimizing the DF loss ℓ_{df} aligns with optimizing the Direct loss ℓ_{dir} . Due to space limits, we provide a more detailed description of the theorem in Appendix A.

3.3.3 Obtain the Ground Truth w_t . The DF loss requires the ground truth w_t as input. We obtain it via an iterative Lagrangian method.

The method has K iterations in total. At each iteration k , we add a penalty term into the original objective of (SLP) to consider the difference between the variable $\bar{a}$ and the required solution a_t^{Ex} , and utilize a Lagrangian parameter $\lambda_{t,k}$ to control its magnitude. Formally, we define (GetA) as

$$\begin{aligned} & \text{minimize} \quad w_{t,k}^\top \bar{a} + \lambda_{t,k} \cdot \|\bar{a} - a_t^{\text{Ex}}\|_2^2 \\ & \text{subject to} \quad (\text{Validness}), (\text{Edge Capacity}), (\text{Node Capacity}) \text{ at } t. \end{aligned} \quad (\text{GetA})$$

Let the solution be $\bar{a}_k$. Then, based on the difference between $\bar{a}_k$ and the required a_t^{Ex} , $w_{t,k}$ and $\lambda_{t,k}$ will get updated by gradient descent and ascent with learning rate γ .

$$w_{t,k+1} = w_{t,k} - \gamma \cdot \bar{a}_k, \quad \lambda_{t,k+1} = \lambda_{t,k} + 2\gamma \cdot \|\bar{a}_k - a_t^{\text{Ex}}\|_2. \quad (\text{GDA})$$

Finally, $w_{t,K+1}$ is used as the ground truth w_t in the DF loss. Our ablation studies show that the choice of K and γ has slight influence on performance, and thus does not require extensive tuning.

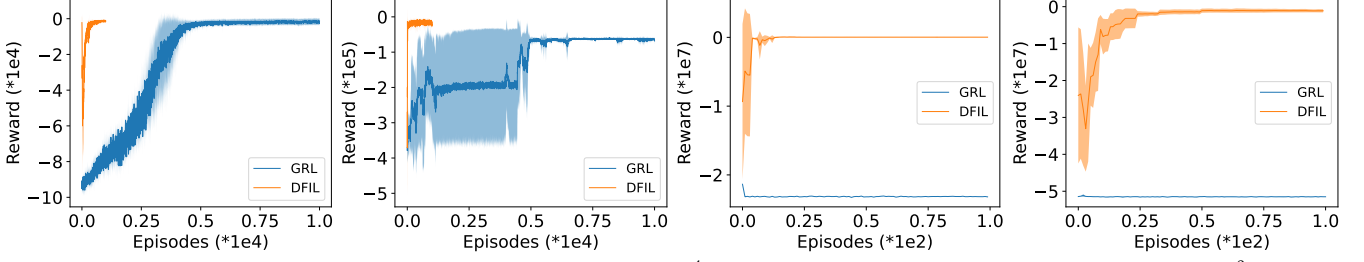


Figure 4: Training curves in SCIM. Left two: 1f2s and 2f10s with 10^4 episodes. Right two: 10f100s and 100f1000s with 10^2 episodes.

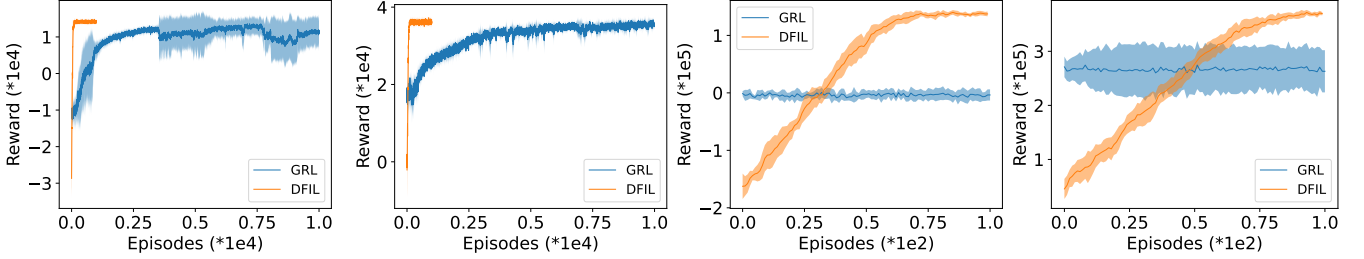


Figure 5: Training curves in DVR. Left two: NY-S and SHEN-S with 10^4 episodes. Right two: NY-M and SHEN-M with 10^2 episodes.

3.3.4 Overall Algorithm. Alg. 1 summarizes our training algorithm. After the interaction with the system for a complete T time steps, we first utilize the realized variables to solve (OHO) for the expert actions. Subsequently, we approximate w iteratively with (GDA) and (GetA). Finally, we update policy π_{NN} with the DF loss function. The algorithm terminates upon the convergence of the policy.

Remark on Stability. While the nested Lagrangian step could raise concerns about convergence or sensitivity to hyper-parameters, our method is empirically both fast and stable. As shown later in Fig. 4-6 and Fig. 8, the algorithm converges smoothly within a small number of episodes and shows minimal sensitivity to K and γ .

4 Application I: SCIM

Setting. We consider four scales: 1f2s, 2f10s, 10f100s, and 100f1000s. They contain 1, 2, 10, 100 factories, and 2, 10, 100, 1000 merchants, respectively. In each system, every factory is connected to every merchant, while there are no edges between factories or between merchants. The action dimension in each system is 3, 22, 1010, and 100100. Detailed settings are shown in Appendix B.

Baselines and Metrics. We consider the following baseline algorithms. **Random.** Random algorithm assigns the expected state of each node in the next step randomly, and solve a constrained problem to reach such expected state to get action, as done in [10]. **Heuristic.** We use the prototypical S-type policy [20] as the heuristic. **Greedy.** Greedy makes decisions by solving a myopic constrained optimization problem. **GRL.** GRL is the SOTA algorithm for SCIM [10]. In every system, each learning-based algorithm is trained using five distinct seeds that are kept consistent across algorithms.

Result Comparisons. Table 1 presents the improvement ratio of long-term profits compared to Random. Fig. 4 compares the training curves. Heuristic and Greedy both perform poor, as Heuristic is sensitive to hyperparameters, and the optimal hyperparameters are hard to find. Greedy also performs poorly as it is myopic and

Table 1: Testing results in 100 episodes in SCIM. Each entry represents the improvement ratio compared to Random.

System	Heuristic	Greedy	GRL	DFIL
1f2s	78.78%	-383.22%	94.20%	99.30%
2f10s	-7.79%	-591.76%	10.14%	71.53%
10f100s	-150.22%	-321.28%	-321.60%	100.50%
100f1000s	-349.34%	-184.22%	-205.85%	91.81%

Table 2: Testing results in 100 episodes in DVR.

System	Heuristic	Greedy	GRL	DFIL
NY-S	72.74%	98.08%	107.58%	108.95%
SHEN-S	195.76%	217.31%	253.69%	251.91%
NY-M	1869.32%	1937.53%	-155.70%	2005.88%
SHEN-M	25.93%	24.19%	-6.35%	31.59%
NY-L	1564.91%	1601.95%	-110.75%	1686.01%
SHEN-L	25.76%	23.69%	-17.10%	30.59%

ignores the future states. In small scale systems, GRL demonstrates satisfactory performance upon convergence. However, as the scale increases, GRL becomes worse and even fail to train. This failure is due to the large action dimensions, making the pivotal exploration in RL becomes difficulty. Our DFIL is approximately 9x and 100x faster than GRL in 1f2s and 2f10s, and it converges in only 10 and 25 episodes in 10f100s and 100f1000s, while GRL completely fails.

5 Application II: DVR

Setting. DVR considers two cities: New York (NY), USA, and Shenzhen (SHEN), China, each with three scales. The small scale is based on real-world road and order datasets, while the middle and large scales are derived from these datasets. The NY systems are NY-S, NY-M, and NY-L, with action dimensions of 56, 616, and 2856. The SHEN systems are SHEN-S, SHEN-M, and SHEN-L, with action dimensions of 68, 748, and 6868. Detailed settings are shown in Appendix C.

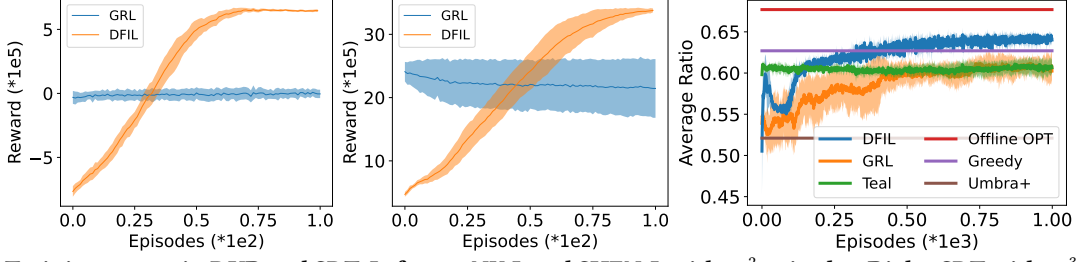


Figure 6: Training curves in DVR and SDT. Left two: NY-L and SHEN-L with 10^2 episodes. Right: SDT with 10^3 episodes.

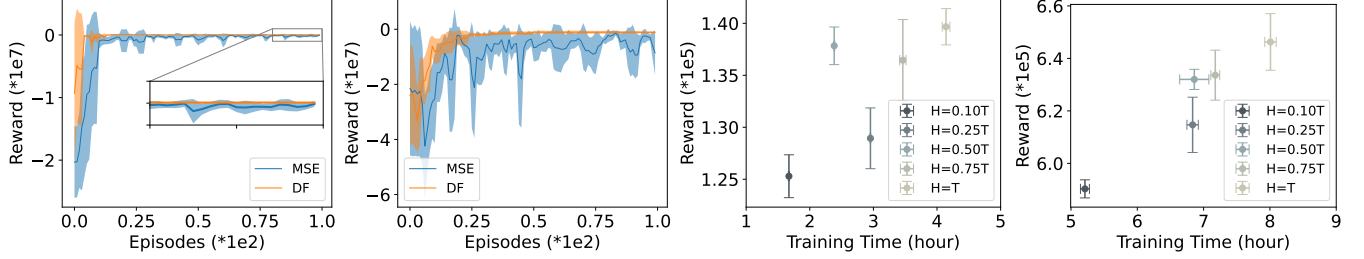


Figure 7: Left two: 10f100s and 100f1000s with different loss functions. Right two: NY-M and NY-L with different H in (OHO).

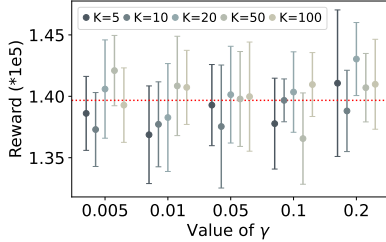


Figure 8: NY-M with different K and γ in (GDA). The red dashed line represents the results with $K = 10$ and $\gamma = 0.1$.

Baselines and Metrics. Random, Greedy and GRL are the same as in SCIM. Heuristic aims to assign a fixed desired number of idle vehicles among all stations [10]. All actions are rounded to integers finally to ensure feasibility, as the number of vehicles must be represented as whole numbers.

Result Comparisons. Table 2 presents the testing results, Fig. 5 and 6 compare the training curves. The testing results show that both Heuristic and Greedy perform well across different scales. This is because, in DVR, each station inherently has a certain number of idle vehicles available to serve passengers. While dispatching additional vehicles from other stations can increase profits, such improvement is limited due to the scheduling costs. As a result, effectively utilizing a station's own vehicles provides a baseline profit, whereas arbitrary vehicle dispatching, as in Random, leads to significant profit losses. In small-scale systems, GRL performs well and even achieves the best results in SHEN-S. However, in medium- and large-scale systems, GRL fails completely during training. Regarding convergence speed, DFIL outperforms GRL by approximately 30x on NY-S and 100x on SHEN-S. Additionally, DFIL achieves convergence within only 100 episodes across all four medium- and large-scale systems, significantly surpassing GRL.

6 Application III: SDT

Setting. We consider the system that contains 2371 satellites and 1 ground station. Each satellite has 4 candidate paths to the ground station. Hence, the dimension of the actions is 9484. The

simulator that simulates the satellite movement, data observation, link disruption, and data transmission are based on real-world datasets, and the simulator will be open-sourced upon acceptance.

Baselines and Metrics. Greedy and GRL are the same for SCIM. Teal [24] is similar to GRL, which firstly outputs the raw actions, and then uses ADMM to modify actions to ensure constraint satisfaction. Offline OPT is the value of (OHO) that represents the performance upper bound. Umbra is the SOTA for satellite data transmission [19], which is non-learning based, and only considers SGLs and ignores the link disruptions. We additionally consider the ISLs, thus named as Umbra+. Detailed settings are shown in Appendix D.

Result Comparisons. Fig. 6 compares the training curves of algorithms. While still trailing Offline OPT, our DFIL outperforms baselines with stable convergence. Umbra+ performs worst as it ignores potential link disruptions, requiring action modifications and clippings to satisfy link capacity constraints, which degrades its performance heavily.

7 Ablations, Hyper-Parameters and Discussions

MSE VS DF Loss. The left two of Fig. 7 show that MSE yields larger fluctuations and lower returns than DF. This is because MSE penalizes weight errors even when they barely change the induced action, introducing unnecessary gradient noise, whereas DF penalizes errors by their action impact.

H in (OHO). The right two of Fig. 7 compare the performance and training time in NY-M and NY-L under different H in (OHO). As H increases, performance improves while training time also rises significantly. This is because a larger H makes (OHO) more closely approximate a full-horizon problem, leading to solutions closer to the optimal. However, larger H also introduces more variables, constraints, and nested transitions, significantly increasing solving time. Interestingly, the trends in performance and time are not linear, showing some fluctuations. We suspect this is because the solving latency and optimality of (GetA) also get changed according

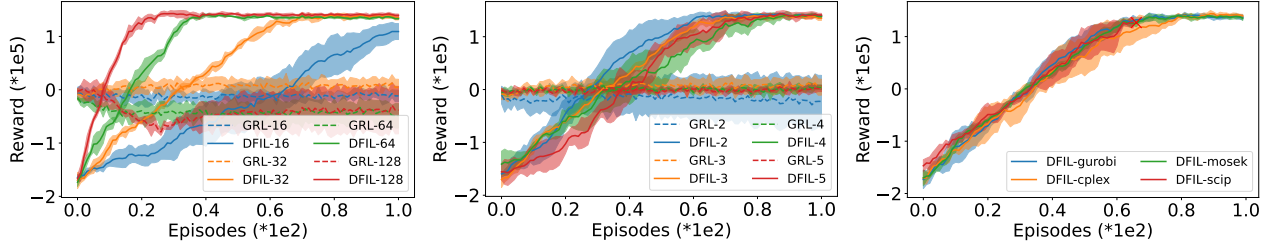


Figure 9: Ablation study in NY-M. From left to right: hidden size, model depth and solvers.

to the optimality of expert actions under different H . We plan to explore this in depth in the future.

K and γ in (GDA). Fig. 8 also compares the results in NY-M under different K and γ . The red dashed line represents the results in Sec. 5. The comparison shows that DFIL is not sensitive to K and γ , with minor fluctuations, indicating that extensive hyperparameter tuning is unnecessary. It also suggests that more iterations (i.e., larger K) generally improve results, though not always.

Models and Solvers. We also conduct ablation study on model hidden size, depth and the solver in use. The default models used in NY-M are 3-layer relu network with hidden size 32, and the solver is Gurobi. For the hidden size, we select 16, 32, 64, and 128. Results in Fig. 9 show that DFIL consistently converges, and increasing the hidden size accelerates the convergence. For the depth, we select 2, 3, 4, and 5 layers. Results in Fig. 9 indicate that increasing the depth does not significantly impact the training. For the solver, we select Gurobi, Mosek, CPLEX, and SCIP as solvers. Results in Fig. 9 show that the choice of solver has minimal impact on training. However, when SCIP is used, the Lagrangian step gets stuck due to a solver exception. We think this may be related to SCIP’s being tailored for IP problems, which is less efficient for other problems and may fail due to numerical instability.

Training and Inference Time. Table 8 reports the total training time, convergence time, and single-inference latency for different system scales, measured on standard, non-specialized computing resources. These results show that our method maintains practical training and inference delays for real-world deployment, and the considered scales already match the application scenarios.

8 Related Works

Networked Optimizations. Our work is orthogonal to the classical networked optimization problems such as maximum flow [8, 12], as CNCP focuses on long-term reward optimization with stochastic demands. Time-expanded network is often adopted for long-term optimization, however, it requires the knowledge of the system dynamics [9, 14] and often faces the curse of dimensionality. To address stochasticity, common methods in the literature often involve making distributional assumptions, such as utilizing Gaussian distributions, to enable analytic tractability [3]. Other approaches include re-planning to adjust schedules [17] or employing sampling-based techniques [1]. However, they have incomplete coverage of state and action spaces, and sampling-based techniques can also be computationally expensive [10]. As for the SOTA works [10, 18], their limitations of scalability have been analyzed in Sec. 2.3 and validated in our experiments of large-scale systems.

Sequential Decision-Making under Constraints. The CNCP we studied also falls under the category of constrained sequential decision-making, but it differs from the popular problems such as CMDP [2], MDP with risk-aware constraints [25, 26], and MDP with chance constraints [4]. This is because our CNCP imposes constraints on the action at each time step, which is more strict, as opposed to the constraints defined on agent’s entire trajectories in the latter problems. Moreover, for the works [6, 23] that also impose constraints for actions at each time step and achieve satisfactory performance in domains like physics and autonomous driving, we fail to make their diffusion models work in CNCP. As analyzed in Sec. 2.3, we believe the main reason is that the constraints in CNCP are coupled and involve decisions across multiple nodes, which makes the actions highly sharp and non-smooth across dimensions.

Decision-Focused Learning. Our DFIL is deeply inspired by the key insight of focusing on the final decisions instead of intermediate weights [16]. As the direct loss is non-differentiable, we adapt the classical SPO+ loss [7] for optimization and design a light-weight iterative Lagrangian process to obtain the necessary inputs. Although some works have also applied decision-focused learning methods to sequential decision-making problems [21, 22], the problems they address are significantly different from ours. Their focus is on the process that involves predicting the transitions from features and then deriving optimal policies from the predicted transitions. In contrast, our work adopts the decision-focused method specifically to solve CNCP. The key distinction lies in that CNCP fundamentally does not require prediction—there is no prediction ground truth nor an explicit single-step constrained optimization problem. Consequently, existing decision-focused learning methods cannot be directly applied. Instead, we must define the π_{NN} ’s outputs, obtain their ground truth, and design a single-step constrained optimization problem that is feasible, efficient to solve, and friendly to decision-focused loss design. These contributions, detailed in Sec. 3.2 and 3.3, represent one of our paper’s core innovations.

9 Conclusion

This paper motivates and initiates the study of the general CNCP, proposes a two-stage policy and a DFIL algorithm for solving, showing its scalability in several large-scale systems. This work serves as a solid starting point, and in the future, we will focus on problems that are more aligned with real-world applications to handle emergencies, enhance robustness, and improve interpretability.

Acknowledgments

This work was supported by NSF China (No. 62372288, W2412089), and Fund USCAST2023-13.

References

- [1] Andrzej Ruszczyński Alexander Shapiro, Darinka Dentcheva. 2021. *Lectures on Stochastic Programming: Modeling and Theory*. SIAM.
- [2] Eitan Altman. 1999. *Constrained Markov decision processes: stochastic modeling*. Routledge.
- [3] Karl J. Astrom. 1971. *Introduction to Stochastic Control Theory*. Elsevier.
- [4] Yinlam Chow, Mohammad Ghavamzadeh, Lucas Janson, and Marco Pavone. 2018. Risk-constrained reinforcement learning with percentile risk criteria. *JMLR* (2018).
- [5] IBM ILOG Cplex. 2009. V12. 1: User's Manual for CPLEX. *International Business Machines Corporation* (2009).
- [6] Franck Djeumou, Thomas Jonathan Lew, NAN DING, Michael Thompson, Makoto Suminaka, Marcus Greiff, and John Subosits. 2024. One Model to Drift Them All: Physics-Informed Conditional Diffusion Model for Driving at the Limits. In *CoRL*.
- [7] Adam N. Elmachtoub and Paul Grigas. 2017. Smart "Predict, then Optimize". *ArXiv* (2017).
- [8] L. R. Ford and D. R. Fulkerson. 1956. Maximal Flow Through a Network. *Canadian Journal of Mathematics* (1956).
- [9] Lester Randolph Ford and Delbert Ray Fulkerson. 1958. Constructing Maximal Dynamic Flows from Static Flows. *Operations Research* (1958).
- [10] Daniele Gammelli, James Harrison, Kaidi Yang, Marco Pavone, Filipe Rodrigues, and Francisco C Pereira. 2023. Graph Reinforcement Learning for Network Control via Bi-Level Optimization. In *ICML*.
- [11] Gurobi Optimization, LLC. 2023. Gurobi Optimizer Reference Manual. <https://www.gurobi.com>
- [12] Frederick Hillier and Gerald Lieberman. 1969. *Introduction To Operations Research*. McGraw-Hill Professional.
- [13] Jonathan Ho, Ajay Jain, and Pieter Abbeel. 2020. Denoising diffusion probabilistic models. In *NeurIPS*.
- [14] D. R. Fulkerson Lester Randolph Ford. 1962. *Flows in Networks*. Princeton University Press.
- [15] Yuanjie Li, Hewu Li, Wei Liu, Lixin Liu, Wei Zhao, Yimei Chen, Jianping Wu, Qian Wu, Jun Liu, Zeqi Lai, and Han Qiu. 2023. A Networking Perspective on Starlink's Self-Driving LEO Mega-Constellation. In *MobiCom*.
- [16] Jayanta Mandi, James Kotary, Senne Berden, Maxime Mulamba, Victor Bucarey, Tias Guns, and Ferdinando Fioretto. 2024. Decision-focused learning: Foundations, state of the art, benchmark and future opportunities. *JAIR* (2024).
- [17] Warren B. Powell. 2022. *Reinforcement Learning and Stochastic Optimization: A Unified Framework for Sequential Decisions*. John Wiley & Sons.
- [18] Aaryan Singhal, Daniele Gammelli, Justin Luke, Karthik Gopalakrishnan, Dominik Helmreich, and Marco Pavone. 2024. Real-time Control of Electric Autonomous Mobility-on-Demand Systems via Graph Reinforcement Learning. In *ECC*.
- [19] Bill Tao, Maleeha Masood, Indranil Gupta, and Deepak Vasishth. 2023. Transmitting, Fast and Slow: Scheduling Satellite Traffic through Space and Time. In *MobiCom*.
- [20] B. Van Roy, D.P. Bertsekas, Y. Lee, and J.N. Tsitsiklis. 1997. A neuro-dynamic programming approach to retailer inventory management. In *CDC*.
- [21] Kai Wang, Sanket Shah, Haipeng Chen, Andrew Perrault, Finale Doshi-Velez, and Milind Tambe. 2021. Learning mdps from features: Predict-then-optimize for sequential decision making by reinforcement learning. In *NeurIPS*.
- [22] Kai Wang, Shresth Verma, Aditya Mate, Sanket Shah, Aparna Taneja, Neha Madhiwalla, Aparna Hegde, and Milind Tambe. 2023. Scalable Decision-Focused Learning in Restless Multi-Armed Bandits with Application to Maternal and Child Health. In *AAAI*.
- [23] Long Wei, Peiyan Hu, Ruiqi Feng, Haodong Feng, Yixuan Du, Tao Zhang, Rui Wang, Yue Wang, Zhi-Ming Ma, and Tailin Wu. 2024. DiffPhyCon: A Generative Approach to Control Complex Physical Systems. In *NeurIPS*.
- [24] Zhiying Xu, Francis Y. Yan, Rachee Singh, Justin T. Chiu, Alexander M. Rush, and Minlan Yu. 2023. Teal: Learning-Accelerated Optimization of WAN Traffic Engineering. In *SIGCOMM*.
- [25] Qisong Yang, Thiago D. Simão, Simon H Tindemans, and Matthijs T. J. Spaan. 2021. WCSAC: Worst-Case Soft Actor Critic for Safety-Constrained Reinforcement Learning. In *AAAI*.
- [26] Zhaoxing Yang, Haiming Jin, Yao Tang, and Guiyun Fan. 2024. Risk-Aware Constrained Reinforcement Learning with Non-Stationary Policies. In *AAMAS*.

A Consistency of DF Loss

Our analysis adapts the analysis given in [7] with our settings and notations. Let $\mathcal{D}$ denote the distribution of (s, a^{Ex}, w) . Then, the population version of the direct risk minimization is

$$\text{minimize}_{\pi_{\text{NN}}} \mathbb{E}_{(s, a^{\text{Ex}}, w) \sim \mathcal{D}} \left[\ell_{\text{dir}}(\pi_{\text{NN}}(s), w) \right]. \quad (1)$$

Table 3: Instantiation on three real-world applications.

Applications	SCIM	DVR	SDT
Node	Factory node, Merchant node	City station	Satellite, Ground station
Edge	Road	Road	ISL, SGL
Time-varying edge set	No	No	Yes
Entity	Goods	Idle vehicles	Observation data packets
Consumption	Customer demands	Passenger orders	Ground applications
Candidate path set	Shortest path	Shortest path	Several shortest paths
Node capacity	Yes	No	No
Edge capacity	No	No	Yes
Control production	Yes	No	No
Control transportation	Yes	Yes	Yes

The population version of the DF risk minimization is

$$\text{minimize}_{\pi_{\text{NN}}} \mathbb{E}_{(s, a^{\text{Ex}}, w) \sim \mathcal{D}} \left[\ell_{\text{df}}(\pi_{\text{NN}}(s), w) \right]. \quad (2)$$

The Fisher consistency is defined as follows.

Definition A.1 (Fisher Consistency). A loss ℓ is said to be Fisher consistent to the Direct loss if $\arg \min_{\pi_{\text{NN}}} \mathbb{E}_{(s, a^{\text{Ex}}, w) \sim \mathcal{D}} \left[\ell(\pi_{\text{NN}}(s), w) \right]$ also minimizes (1).

Then, we have the following theorem on the Fisher consistency.

THEOREM A.2. For all s , assume that the distribution of $w|s$ is centrally symmetric about its mean $\mathbb{E}[w|s]$, and the distribution of $w|s$ is continuous. Furthermore, assume the interior of the feasible region of each time step is nonempty. Then, any minimizer of the DF risk (2) is also a minimizer of the Direct risk (1). That is, the DF loss is Fisher consistent with respect to the Direct loss.

The key idea is that the DF loss is specifically constructed as a convex, differentiable upper bound on the Direct loss, and its minimization is tightly aligned with minimizing the Direct loss due to the way it approximates the regret function. By proving that the expected DF loss is minimized at the true parameters, one can establish that the DF loss is Fisher consistent with respect to the Direct loss. We omit the proof here and recommend readers the Sec. 4.1 in the original work [7] for more details.

B SCIM

B.1 Problem formulation

The supply chain system is expressed by a network $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V} = \mathcal{V}_f \cup \mathcal{V}_m$. $\mathcal{V}_f$ is the set of factory nodes, and $\mathcal{V}_m$ is the set of merchant nodes. $\mathcal{E} = \{(i, j) | i \in \mathcal{V}_f, j \in \mathcal{V}_m\}$ denotes the edge set from each factory node i to each merchant node j . The path set $\mathcal{M}_{i,j} = \{(i, j) \in \mathcal{E}\}$, i.e., it equals the shortest path between nodes. At each step t , demand d_t^i emerges randomly at each node $i \in \mathcal{V}_m$. Unsatisfied demand will maintain over time and are represented as a negative stock. Each node i has a storage limit c^i , such that the current quantity of goods x_t^i cannot exceed it. Satisfied demands will generate revenue, while production, storage, negative stock and transportation will generate expenses.

Table 4: Para. in 1f2s.

Parameter	Value	Parameter	Value
Maximum demand d_{max}	[2, 16]	Maximum demand d_{max}	[2, 2, 2, 10, 10, 10, 18, 18]
Demand variance d_{var}	[2] $_{i \in \mathcal{V}}$	Demand variance d_{var}	[2] $_{i \in \mathcal{V}}$
Episode length T	30	Episode length T	30
Production time	[1] $_{i \in \mathcal{V}_f}$	Production time	[1] $_{i \in \mathcal{V}_f}$
Transportation time	[1] $_{i,j \in \mathcal{V}}$	Transportation time	[1] $_{i,j \in \mathcal{V}}$
Storage capacity	[100] $_{i \in \mathcal{V}_f}, [15]_{i \in \mathcal{V}_m}$	Storage capacity	[100] $_{i \in \mathcal{V}_f}, [15]_{i \in \mathcal{V}_m}$
Storage expense	[1] $_{i \in \mathcal{V}_f}, [2]_{i \in \mathcal{V}_m}$	Storage expense	[1] $_{i \in \mathcal{V}_f}, [2]_{i \in \mathcal{V}_m}$
Production expense	[5] $_{i \in \mathcal{V}_f}$	Production expense	[5] $_{i \in \mathcal{V}_f}$
Negative stock expense	[21] $_{i \in \mathcal{V}_m}$	Negative stock expense	[21] $_{i \in \mathcal{V}_m}$
Transportation expense	[0.3] $_{i,j \in \mathcal{V}}$	Transportation expense	[0.3] $_{i,j \in \mathcal{V}}$
Price p	[15] $_{i \in \mathcal{V}_m}$	Price p	[15] $_{i \in \mathcal{V}_m}$
Action dimension	3	Action dimension	22

Table 5: Para. in 2f10s.

Table 6: Para. in 10f100s. Table 7: Para. in 100f1000s.

Parameter	Value	Parameter	Value
Maximum demand d_{max}	[30, 40, 60]	Maximum demand d_{max}	[30, 40, 60]
Demand variance d_{var}	[5] $_{i \in \mathcal{V}_m}$	Demand variance d_{var}	[5] $_{i \in \mathcal{V}_m}$
Episode length T	30	Episode length T	10
Production time	[1] $_{i \in \mathcal{V}_f}$	Production time	[1] $_{i \in \mathcal{V}_f}$
Transportation time	[1] $_{i,j \in \mathcal{V}}$	Transportation time	[1] $_{i,j \in \mathcal{V}}$
Storage capacity	[1000] $_{i \in \mathcal{V}_f}, [50]_{i \in \mathcal{V}_m}$	Storage capacity	[1000] $_{i \in \mathcal{V}_f}, [150]_{i \in \mathcal{V}_m}$
Storage expense	[1] $_{i \in \mathcal{V}_f}, [2]_{i \in \mathcal{V}_m}$	Storage expense	[1] $_{i \in \mathcal{V}_f}, [2]_{i \in \mathcal{V}_m}$
Production expense	[5] $_{i \in \mathcal{V}_f}$	Production expense	[1] $_{i \in \mathcal{V}_f}$
Negative stock expense	[21] $_{i \in \mathcal{V}_m}$	Negative stock expense	[21] $_{i \in \mathcal{V}_m}$
Transportation expense	[0.3] $_{i,j \in \mathcal{V}}$	Transportation expense	[0.3] $_{i,j \in \mathcal{V}}$
Price p	[15] $_{i \in \mathcal{V}_m}$	Price p	[30] $_{i \in \mathcal{V}_m}$
Action dimension	1010	Action dimension	100100

The system controller needs to decide at each time step the production at each factory node and the transportation between nodes, in order to maximize the long-term profits. Formally, the goal of SCIM is to solve the following problem.

$$\begin{aligned}
& \text{maximize} \quad \mathbb{E} \left[\sum_{t=1}^T \sum_{i \in \mathcal{V}} r_t^i - e_t^i \right] \\
& \text{subject to} \quad 0 \leq x_t^i \leq c^i, \forall i \in \mathcal{V}, t, \\
& \quad 0 \leq x_t^i \cdot \sum_{(i,j) \in \mathcal{E}} f_t^{i,j} \leq x_t^i, \forall i \in \mathcal{V}_f, t.
\end{aligned} \tag{SCIM}$$

B.2 OHO solver

Given buffer $\mathcal{B}$ that contains the realized demands $\{d_t^i\}$ of each node i at each step t in an episode, the OHO solver solves the following problem for each step $t \in \mathcal{T}$ to obtain the expert actions:

$$\begin{aligned}
& \max_{f_t^{i,j}, p_t^{i,j}, t \leq t' \leq t+H} \sum_{t'=t}^{t'+H} \sum_{i \in \mathcal{V}} r_{t'}^i - e_{t'}^i, \\
& \text{s.t.} \quad 0 \leq x_{t'}^i \leq c^i, \\
& \quad 0 \leq x_{t'}^i \cdot \sum_{(i,j') \in \mathcal{E}} f_{t'}^{i,j'} \leq x_{t'}^i, \\
& \quad x_{t'+1}^i = x_{t'}^i + p_{t'+1-\tau}^i - x_{t'}^i \cdot \sum_{(i,j') \in \mathcal{E}} f_{t'}^{i,j'}, \forall i \in \mathcal{V}_f, \\
& \quad x_{t'+1}^i = x_{t'}^i - d_{t'}^i + x_{t'}^{j'} \cdot \sum_{(j',i) \in \mathcal{E}} f_{t'}^{j',i}, \forall i \in \mathcal{V}_m, t \leq t' \leq t+H.
\end{aligned} \tag{SCIM OHO}$$

The problem can be solved with SOTA solvers, such as Gurobi [11] and CPLEX [5]. In the problem, $0 < H \leq T$ is the hyper-parameter to balance the optimality and efficiency. We also note that for each time step t , although the OHO problem contains variables from t to $t+H$, we ultimately only use the solved variables of time t as the expert actions of time t . In the end, we need to solve T OHO problems that start at different time steps.

B.3 Problem parameter settings

For the demand, we assume each merchant has an independent demand-generating process following [10]: a seasonal demand behavior, which is defined as follows.

$$d_t^i = \left\lfloor \frac{d_{max}^i}{2} \left(1 + \cos \frac{4\pi(2i+t)}{T} \right) \right\rfloor + U(0, d_{var}^i).$$

where d_{max}^i and d_{var}^i are the maximum demand value and variance, respectively.

Parameters of different system scales are defined in Table 4-7.

Table 8: Training time and inference time with 16 vCPU Intel(R) Xeon(R) Platinum 8481C and a RTX 4090D (24GB) card.

	NY-M	NY-L	SCIM-1f2s	2f10s	10f100s	100f1000s
Action dim	616	2856	3	22	1010	100100
Training time of all episodes	3.0h	6.0h	1.2h	3.5h	12.1h	40.0h
Training time till convergence	2.3h	4.5h	0.4h	0.4h	1.5h	10.0h
Each inference time	0.2s	0.8s	0.1s	0.1s	1.2s	3.1s

B.4 Neural network and training settings

In SCIM, each node's local state is 5-d: the current quantity of goods, the user demand prediction for 3 look-ahead timesteps (same as the setting of [10]), the arriving goods in the next 3 timesteps, the completed production in the next 3 timesteps, the one-hot time step index. Each edge between factory node and merchant node has 2-d features: transportation time and expense.

Each baseline algorithm for SCIM uses 1 EdgeConv layer to extract latent embeddings. The π_{NN} policy of DFIL, and policy and critic of GRL all contain 2-layer linear networks, with hidden dim 256 and activation function relu. The optimizer used is Adam, with learning rate 0.00005. The Lagrangian iterations in (GDA) is 10, and the learning rate γ is 0.1.

B.5 Hardware

Regarding the hardware used in our experiments: All experiments were conducted on an Intel(R) Xeon(R) CPU E5-2650 v3 (2.30GHz base frequency, max 20 threads), with learning-based algorithms trained and tested on a single NVIDIA GeForce RTX 2080 Ti (8GB) card. While these specifications are not high-end, they were sufficient for our study. In addition, we re-train and re-test our DFIL on higher-performance hardware, and summarize the training time and inference time in Table 8.

C DVR

C.1 Problem formulation

In DVR, the city is partitioned into multiple stations, with roads connecting adjacent, accessible stations. Let network $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ represent the stations and their connections. Passenger orders d_t^i randomly emerge at each station i during each step t , awaiting pick-up and delivery to their respective destinations. The order value is the system's revenue. Idle vehicles can be dispatched by the system controller to other stations to anticipate and serve the passengers there in the future, with certain dispatch expense. Furthermore, as the dispatching is achieved between neighboring stations, the path set $\mathcal{M}_{i,j}$ between any node pair (i, j) contains only the shortest road between i and j .

Table 9: Stats. of NY-S.

Parameter	Value
Number of stations	14
Number of roads	56
Number of demands	4830
Price of demands: average	29.62
Price of demands: std	15.16
Price of demands: max	75.60
Price of demands: min	7.50
Travel time of demands: average	19.42
Travel time of demands: std	8.87
Travel time of demands: max	68.00
Travel time of demands: min	5.00
Rebalance time of vehicles: average	14.49
Rebalance time of vehicles: std	7.09
Rebalance time of vehicles: max	33.40
Rebalance time of vehicles: min	1.00
Number of vehicles	1500
Action dimension	56

Table 10: Stats. of SHEN-S.

Parameter	Value
Number of stations	17
Number of roads	68
Number of demands	24375
Price of demands: average	26.19
Price of demands: std	16.36
Price of demands: max	128.50
Price of demands: min	10.00
Travel time of demands: average	16.56
Travel time of demands: std	9.46
Travel time of demands: max	66.00
Travel time of demands: min	3.00
Rebalance time of vehicles: average	15.26
Rebalance time of vehicles: std	8.00
Rebalance time of vehicles: max	35.29
Rebalance time of vehicles: min	1.00
Number of vehicles	1777
Action dimension	68

As a system controller, his decision for the system is to optimally dispatch idle vehicles among these stations. This decision involves a balance between servicing as many passenger orders as possible and avoiding excessive dispatch expenses, all with the goal of maximizing the system's long-term profits. Formally, the goal of DVR is to solve the following problem. Note that each $f_t^{i,j}$ need to be clipped into integer afterwards to keep valid meaning, i.e., the number of dispatched idle vehicles.

$$\begin{aligned}
& \text{maximize} \quad \mathbb{E} \left[\sum_{t=1}^T \sum_{i \in \mathcal{V}} r_t^i - e_t^i \right] \\
& \text{subject to} \quad 0 \leq x_t^i \cdot \sum_{(i,j) \in \mathcal{E}} f_t^{i,j} \leq x_t^i, \forall i \in \mathcal{V}, t.
\end{aligned} \tag{DVR}$$

C.2 OHO solver

Given buffer $\mathcal{B}$ that contains the realized passenger orders $\{d_t^i\}$ and the number of vehicles becoming active (i.e., offline to online) and available (i.e., busy to idle) $\{n_t^i\}$ of each site i at each step t , the OHO solver solves the following problem for each step $t \in \mathcal{T}$ to obtain the expert decision:

$$\begin{aligned}
& \max_{f_t^{i,j}, t \leq t' \leq t+h} \sum_{t'=t}^{t'+h} \sum_{i \in \mathcal{V}} r_{t'}^i - e_{t'}^i, \\
& \text{s.t.} \quad 0 \leq x_{t'}^i \cdot \sum_{(i,j') \in \mathcal{E}} f_{t'}^{i,j'} \leq x_{t'}^i, \\
& \quad x_{t'+1}^i = x_{t'}^i - d_{t'}^i + n_{t'+1}^i + x_{t'}^{j'} \cdot \sum_{(j',i) \in \mathcal{E}} f_{t'}^{j',i} \\
& \quad - x_{t'}^i \cdot \sum_{(i,j') \in \mathcal{E}} f_{t'}^{i,j'}, \forall i, t \leq t' \leq t+h.
\end{aligned} \tag{DVR OHO}$$

Similarly, the above problem can be solved with SOTA solvers.

C.3 Datasets

The NY-S and SHEN-S contain the real-world datasets collected by [10], and the statistics about the datasets are shown in Table 9-10. Note that they are very small-scale systems, with only 56 and 68 dimension of action, respectively. Based on these two datasets, we scale up the systems by repeating the topology of stations and roads to create NY-M, NY-L, SHEN-M and SHEN-L, which have action dimension of 616, 2856, 748 and 6868, respectively.

C.4 Neural network and training settings

In DVR, each node's local state is 4-d: the current number of vehicles, the sum of the user demands multiplied by the price for 3 look-ahead timesteps (same as the setting of [10]), the arriving vehicles in the next 3 timesteps, the one-hot time step index.

Each baseline algorithm for DVR uses 1 GCNConv layer to extract latent embeddings. The π_{NN} policy of DFIL, and policy and critic of GRL all contain 3-layer linear networks, with hidden dim 32 and activation function relu. The optimizer used is Adam, with learning rate 0.001. The Lagrangian iterations in (GDA) is 10, and the learning rate γ is 0.1.

D SDT

D.1 Problem formulation

In SDT, satellites move along the orbits and establish connections with ground stations through inter-satellite links (ISLs) and satellite-to-ground links (SGLs). When passing over specific regions, satellites collect observational data, which will be transmitted via the links to ground stations for monitoring and response. Both ISL and SGL have only limited bandwidth for data transmission.

Let network $\mathcal{G} = (\mathcal{V}, \mathcal{E}_t)$ represent the satellites, the ground stations, and the active links at t . We adopt the classical grid inter-satellite topology, where each satellite has ISLs with the two neighboring satellites in the same orbit, and with the two neighboring satellites in the adjacent orbits. The SGL is established only when the satellite and ground station are within the range threshold. Moreover, ISLs and SGLs may be disrupted due to factors such as atmospheric orbit drags, multi-body orbit perturbations, solar/lunar activities, thermal fluctuations, and space radiations. Hence, the t is subscripted in $\mathcal{E}_t$ to emphasize the time-varying characteristics of the links.

The observational data of volume d_t^i emerge at each satellite i during each step t , awaiting transmission to the ground stations. The transmitted ratio acts as the system's revenue of the CNCP formulation. SDT has no expense. The path set $\mathcal{M}_{t,i,j}$ at each step t between any satellite i and ground station j contains the currently active paths (i.e., a sequence of ISLs and an SGL) from i to j . In experiments, each $\mathcal{M}_{t,i,j}$ contains maximal 4 active paths.

The system controller's decision for the system is to transmit the data from satellites to ground stations under link bandwidth constraints, with the goal of maximizing the system's long-term profits. Formally, let j denote the ground station node. The goal of SDT is to solve the following problem.

$$\begin{aligned}
& \text{maximize} \quad \mathbb{E} \left[\sum_{t=1}^T \sum_{i \in \mathcal{V}} r_t^i \right] \\
& \text{subject to} \quad 0 \leq x_t^i \cdot \sum_{m \in \mathcal{M}_{t,i,j}} f_t^{i,j}(m) \leq x_t^i, \forall i \in \mathcal{V}, t.
\end{aligned} \tag{SDT}$$

D.2 OHO solver

Given buffer $\mathcal{B}$ that contains the realized data volume $\{d_t^i\}$, the OHO solver solves the following problem for each step $t \in \mathcal{T}$ to

obtain the expert decision:

$$\begin{aligned}
 & \max_{f_{t',t \leq t' \leq t+h}^{i,j}} \sum_{t'=t}^{t'=t+h} \sum_{i \in \mathcal{V}} r_{t'}^i, \\
 \text{s.t.} \quad & 0 \leq x_{t'}^i \cdot \sum_{m \in \mathcal{M}_{t',i,j}} f_{t'}^{i,j}(m) \leq x_{t'}^i, \quad (\text{SDT OHO}) \\
 & x_{t'+1}^i = x_{t'}^i + d_{t'}^i - x_{t'}^i \cdot \sum_{m \in \mathcal{M}_{t',i,j}} f_{t'}^{i,j}(m), \forall i, t \leq t' \leq t+h.
 \end{aligned}$$

In the problem, j is the ground station node. The state transition in (SDT OHO) does not consider the data volume increase in the ground station node, as the received data are consumed immediately for downstream usage (e.g., monitoring tasks).

D.3 Simulator

To support policy training and evaluation, we develop a lightweight simulator based on real-world datasets.

Datasets. One of the datasets is the Starlink’s Two-Line Element (TLE) dataset. TLE is the standard data format used by NASA, containing basic information of each satellite, such as its velocity, acceleration, position and altitude. Given history TLE data of a satellite, one can predict its future position accurately with the orbit propagators such as SGP4. Hence, this dataset is used in our simulator to simulate the movement of satellites. The entire dataset contains 41,188,538 TLEs from May 2019 to July 2022, containing data of over 8,000 satellites, open sourced by [15].

Another dataset is the conjunction reports. They are generated by the U.S. Space Surveillance Network to generate alerts for potential collision events and probabilities. Starlink conducts satellite self-maneuvering when the probability is higher than 10^{-5} to keep safe and avoid the collisions. The main consequence of such conservative self-maneuvering is that the ISLs and SGLs established by the satellite will get disrupted. We use such events to simulate the link disrupts. The entire dataset includes 9,350,134 reports between 21,743 pairs of space objects, open sourced by [15].

Workflow. The user firstly specifies the target region to collect observational data, and the simulation horizon. Then, at each step, the simulator conducts the following events.

- Data collection. When each satellite moves over the target region, it will collect the observation data, and store it into the storage.
- Active path update. Based on the collision events happened within the step, influenced satellites will conduct self-maneuvering operations. Hence, their ISLs and SGLs will become disrupted. Further, each path of the candidate path set will also become inactive if the path contains the disrupted ISLs or SGLs.
- Data transmission. Based on the updated candidate path set, each satellite decides the portion of data to transmit over each path. This is the place for policy or other baseline algorithms to make decisions. After that, data is transmitted to ground station according to the assigned portion over each path.
- Satellite movement. Afterwards, each satellite moves along the orbit to the next position, calculated by SGP4 propagator based on historical TLE data of each satellite.

After a complete episode, all the settings are set to the initial value for repeated training. The ratio of transmitted data will be

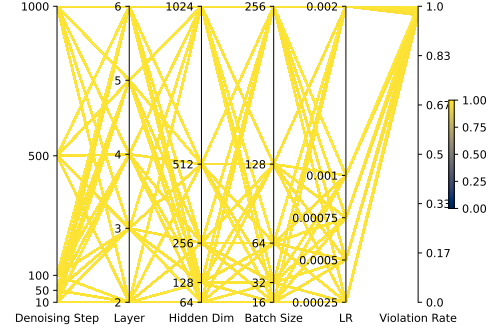


Figure 10: Hyperparameter combinations for the diffusion model.

reported as the metric. In our experiments, there are total 2371 satellites and 1 ground station. Each satellite has maximal 4 active paths to the ground station. Hence, the action dimension is 9484. The training time of DFIL is approximately 10 hours.

D.4 Neural network and training settings

In SDT, each node’s local state includes: the satellite’s ID in one-hot vector, its own features, and current data volume.

GRL, Teal and DFIL all adopt 3-layer ReLU policy networks with a hidden layer size of 128 and a learning rate of 0.0005. GRL and Teal also have an extra 3-layer ReLU critic network with a hidden layer size of 128. Both GRL and Teal use the PPO algorithm, with a clipping size of 0.2 and an entropy coefficient of 0.1. The Lagrangian iterations in (GDA) is 10, and the learning rate γ is 0.2.

E Diffusion Models

Datasets. We use the Random algorithm to generate 100 episodes in the SCIM-10f100s environment. Note that Random will generate feasible actions as it contains the optimization stage to make actions feasible. Each action has dimension 1010. 80 randomly chosen episodes are used for training, and the remaining 20 episodes are used for testing.

Network and Hyperparameters. The module for processing the state is similar to DFIL, where 1 EdgeConv layer is used to extract the latent embeddings. Then, the embeddings, one hot representation of the time step, together with the action of this step, are input to a denoising network. For such a network, we have the following hyperparameter choices.

- Denoising step in $\{10, 50, 100, 500, 1000\}$.
- Number of layers of the ReLU network in $\{2, 3, 4, 5, 6\}$.
- Hidden dimension of the ReLU network in $\{64, 128, 256, 512, 1024\}$.
- Batch size in $\{16, 32, 64, 128, 256\}$.
- Learning rate in $\{0.00025, 0.0005, 0.00075, 0.001, 0.002\}$.

Results. For each possible combination of the hyper-parameters, we train a diffusion model using the DDPM algorithm until convergence. In total, we obtain over 3000 models. Their evaluation results are summarized in Fig. 10. It shows that the diffusion model fails to approximate the distribution of the feasible actions, as they all generates actions with violation rate nearly 100%.